

src\case_studies\control\pid.py

Generated: 2026-02-26 12:33:34

pid.py

src\case_studies\control\pid.py

```
# Local (controlbook)
from .integration_filter import IntegrationFilter

# TODO: should this inherit from ControllerBase and implement ff term + saturation?
class PID:
    """
    PID (Proportional-Integral-Derivative) controller helper class.

    NOTE: This is a HELPER class used inside full controller implementations
    (Like ArmControllerPID). You can either write this once, and then use it in
    each controller (e.g. D_mass/pid_controller.py), or you can implement PID control
    directly inside each controller class instead. You don't need both.

    This class implements PID control law, which combines three actions:
    - Proportional (P): Responds to current error
    - Integral (I): Eliminates steady-state error (accumulates error over time)
    - Derivative (D): Provides damping (responds to rate of change) as either modified
      or regular form.

    Control law:  $u = k_p \cdot error + k_i \cdot integral(error) + k_d \cdot error\_dot$ 
    (or modified form with  $-k_d \cdot ydot$ )

    The derivative of  $y$  (or  $e$ ) must be calculated outside of this class and passed
    in to the update methods. This is typically done using the DirtyDerivativeFilter
    or from direct state access.
    """

    def __init__(
        self,
        kp: float,
        ki: float,
        kd: float,
        ts: float,
        anti_windup_vel_tol: float = float("inf"),
    ):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.ts = ts
        self.integrator = IntegrationFilter(ts)
```

```

self.anti_windup_vel_tol = anti_windup_vel_tol
self.y_ref_prev = 0.0
self.ydot = 0.0 # Store last ydot for plotting

def update(self, y_ref: float, y: float, error_dot: float):
    """
    Returns output of  $u = k_p \text{error} + k_d \text{error\_dot} + k_i \text{error\_integral}$ .

    Args:
        y_ref (float): reference value for signal y.
        y (float): current value of signal y.
        error_dot (float): derivative of error.
    """
    error = y_ref - y

    # reset the integrator if the reference changes
    if self.y_ref_prev != y_ref:
        self.integrator.integral = 0.0
        self.y_ref_prev = y_ref

    if abs(error_dot) > self.anti_windup_vel_tol:
        error_integral = self.integrator.update(0.0)
    else:
        error_integral = self.integrator.update(error)

    u = self.kp * error + self.kd * error_dot + self.ki * error_integral
    return u

def update_modified(self, y_ref: float, y: float, ydot: float):
    """
    Returns output of  $u = k_p \text{error} - k_d y\_dot + k_i \text{error\_integral}$ .

    Args:
        y_ref (float): reference value for signal y.
        y (float): current value of signal y.
        ydot (float): current derivative of signal y.
    """
    self.ydot = ydot # Store for plotting

    # reset the integrator if the reference changes
    if self.y_ref_prev != y_ref:
        self.integrator.integral = 0.0
        self.y_ref_prev = y_ref

    error = y_ref - y
    # if anti_windup_velocity and abs(ydot) > self.anti_windup_vel_tol:
    if abs(ydot) > self.anti_windup_vel_tol:
        error_integral = self.integrator.update(0.0)
    else:
        error_integral = self.integrator.update(error)

    u = self.kp * error - self.kd * ydot + self.ki * error_integral

    return u

# TODO: decide if inheriting from ControllerBase where saturation is known

```

```

# and this could go inside of the update functions

def anti_windup_from_saturation(self, u_sat: float, u_unsat: float):
    if not self.ki == 0.0:
        # self.integrator.add_anti_windup_saturation(self.ki, u_sat, u_unsat)
        correction_amount = (u_sat - u_unsat) / self.ki
        # correction_amount = (u_sat - u_unsat) / abs(self.ki)
        self.integrator.add_anti_windup(correction_amount)
        # self.error_integral += (u_sat - u_unsat) / self.ki

# TODO: should there be a single update function with a flag for modified?

# def update(self, y_ref, y, ydot=None):
#     error = y_ref - y
#     p_term = self.kp * error
#
#     error_integral = self.integrator.update(error)
#     i_term = self.ki * error_integral
#
#     if self.modified:
#         if ydot is None:
#             ydot = self.dirty_derivative.update(y)
#             d_term = -self.kd * ydot
#         else:
#             error_dot = self.dirty_derivative.update(error)
#             d_term = self.kd * error_dot
#
#     u = p_term + i_term + d_term
#     return u

```